



UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Programación Concurrente

Integrantes:

- Cáceres, Juan Manuel
- Ferrando, Enzo Jesús
- Giraudo, Ignacio José
- Miglierini, Carlos Emanuel
- Zárate, Fernando Agustín

Profesor:

Dr. Ing. Luis Orlando Ventre
Ing. Mauricio Ludemann

06/2026

Descripción del sistema:

El sistema modelado representa el núcleo de un Payment Service Provider (PSP), es decir, un proveedor de servicios de pago encargado de recibir, procesar y liquidar transacciones financieras de distintos tipos. Este tipo de sistemas es el que opera detrás de plataformas como Mercado Pago, PayU o cualquier pasarela de cobro que intermedie entre un comercio y las redes bancarias o de tarjetas.

Recursos compartidos:

El sistema tiene dos recursos de uso exclusivo que deben ser coordinados para evitar condiciones de carrera:

- Gateway de Red de Tarjetas (P7): sesión con las redes Visa/Mastercard para solicitar autorizaciones y confirmar capturas de fondos.
- Motor antifraude (P8): servicio de scoring que evalúa el riesgo de cada transacción.

Flujos de procesamiento:

Una vez admitida en el sistema, cada transacción es ruteada a uno de tres flujos:

- Pago con tarjeta (IT1): dos etapas — autorización al issuer y captura del fondo. Requiere el Gateway (P7) durante todo el flujo.
- Pago de alto riesgo (IT2): una etapa de scoring en tiempo real. Requiere simultáneamente el Gateway (P7) y el motor antifraude (P8).
- Transferencia bancaria (IT3): dos etapas — validación de cuenta destino y ejecución de la transferencia. Requiere el motor antifraude (P8) durante todo el flujo.

Ciclo de vida:

Toda transacción sigue el mismo ciclo: arriba al sistema → es admitida → se rutea a un flujo → atraviesa sus etapas → se deposita en el buffer de salida → se liquida y el ciclo se reinicia.

Red de Petri Analizada:

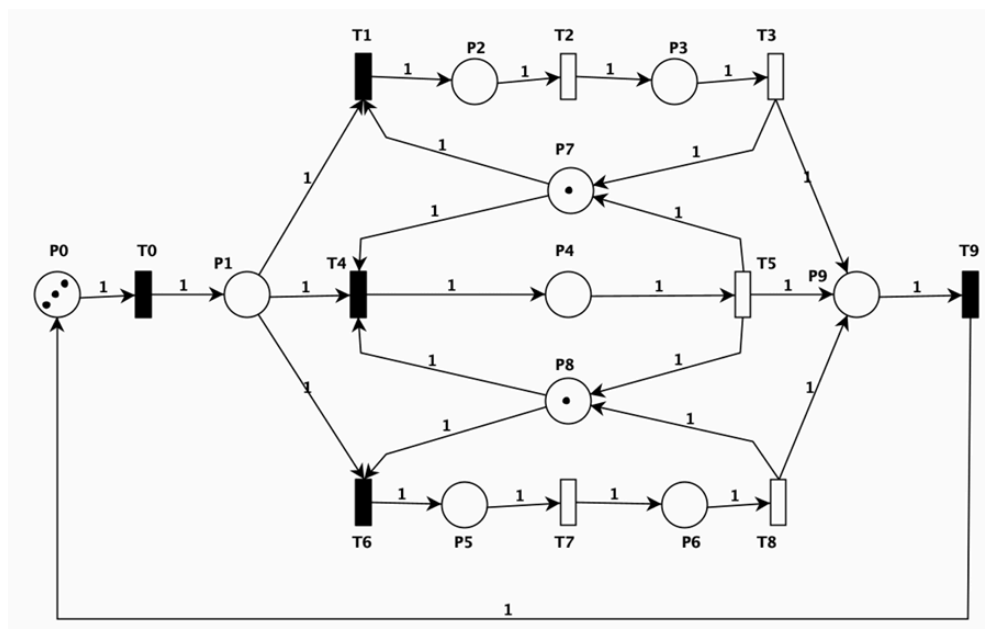


Tabla de estados del sistema:

Estado	Plaza	Descripción
E0	P0	Cola de arribo de transacciones del sistema.
E1	P1	Admitida, validada, autenticada — esperando ruteo.
E2	P2	Solicitud enviada al issuer — esperando respuesta.
E3	P3	Respuesta recibida — capturando fondos.
E4	P4	Scoring antifraude + consulta gateway en curso.
E5	P5	Motor antifraude validando cuenta destino.
E6	P6	Ejecución en red bancaria en curso.
E7	P9	Buffer de salida con las transacciones procesadas, listas para ser confirmadas al cliente y enviadas a liquidación.

Tabla de eventos del sistema:

Evento	Plaza	Descripción	Precondición	Postcondición	α
Arribo	T0	Transacción entra al sistema.	$P0 \geq 1$	$P0-1, P1+1$	0
Ruteo tarjeta	T1	Toma gateway, inicia flujo tarjeta.	$P1 \geq 1, P7 \geq 1$	$P1-1, P7-1, P2+1$	0
Solicitar autorización	T2	Envía solicitud.	$P2 \geq 1$	$P2-1, P3+1$	100ms
Capturar fondos	T3	Confirma captura, libera gateway.	$P3 \geq 1$	$P3-1, P7+1, P9+1$	80ms
Ruteo alto riesgo	T4	Toma gateway y antifraude.	$P1 \geq 1, P7 \geq 1, P8 \geq 1$	$P1-1, P7-1, P8-1, P4+1$	0
Completar evaluación	T5	Finaliza scoring antifraude, libera ambos recursos.	$P4 \geq 1$	$P4-1, P7+1, P8+1, P9+1$	400ms
Ruteo transferencia	T6	Toma motor antifraude	$P1 \geq 1, P8 \geq 1$	$P1-1, P8-1, P5+1$	0
Validar cuenta	T7	Valida cuenta en red bancaria	$P5 \geq 1$	$P5-1, P6+1$	200ms
Ejecutar transferencia	T8	Ejecuta y libera el motor antifraude	$P6 \geq 1$	$P6-1, P8+1, P9+1$	140ms
Liquidar	T9	Entrega al buffer de salida.	$P9 \geq 1$	$P9-1, P0+1$	0

Tabla de recursos del sistema:

<u>Recurso</u>	<u>Plaza</u>	<u>Tokens iniciales</u>
Gateway de Red de Tarjetas.	P7	1
Motor antifraude.	P8	1

Resultados obtenidos de la herramienta PIPE:

Petri net state space analysis results

Bounded	true
Safe	false
Deadlock	false

P-Invariant equations

$$\begin{aligned}M(P2) + M(P3) + M(P4) + M(P7) &= 1 \\M(P4) + M(P5) + M(P6) + M(P8) &= 1 \\M(P0) + M(P1) + M(P2) + M(P3) + M(P4) + M(P5) + M(P6) + \\M(P9) &= 3\end{aligned}$$

Petri net invariant analysis results

T-Invariants

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
1	1	1	1	0	0	0	0	0	1
1	0	0	0	1	1	0	0	0	1
1	0	0	0	0	0	1	1	1	1

The net is covered by positive T-Invariants, therefore it might be bounded and live.

P-Invariants

P0	P1	P2	P3	P4	P5	P6	P7	P8	P9
0	0	1	1	1	0	0	1	0	0
0	0	0	0	1	1	1	0	1	0
1	1	1	1	1	1	1	0	0	1

The net is covered by positive P-Invariants, therefore it is bounded.

Características:

- Bounded (acotada): verdadero. La cantidad de tokens en cada plaza nunca crece indefinidamente. Esto garantiza que el sistema no acumule transacciones sin límite.
- Safe (segura): falso. Algunas plazas pueden tener más de 1 token simultáneamente. Esto es esperable dado que P0 arranca con 3 tokens, por lo que varias transacciones pueden estar en el sistema al mismo tiempo.
- Deadlock: falso. El sistema nunca llega a un estado donde ninguna transición pueda dispararse. La ejecución siempre puede continuar.

P-Invariantes:

Los P-invariantes representan cantidades que se conservan constantes ante cualquier disparo de la red.

- IP1: $M(P2) + M(P3) + M(P4) + M(P7) = 1$
El Gateway de Red de Tarjetas (P7) siempre está en exactamente uno de dos estados: libre (token en P7) o siendo utilizado por un flujo (token en P2, P3 o P4). Garantiza la exclusión mutua del Gateway.
- IP2: $M(P4) + M(P5) + M(P6) + M(P8) = 1$
El motor antifraude (P8) siempre está libre o siendo utilizado por un flujo (token en P4, P5 o P6). Garantiza la exclusión mutua del motor antifraude.
- IP3: $M(P0) + M(P1) + M(P2) + M(P3) + M(P4) + M(P5) + M(P6) + M(P9) = 3$
En todo momento hay exactamente 3 transacciones activas en el sistema, lo que se corresponde con el marcado inicial de 3 tokens en P0.

T-Invariantes:

Los T-invariantes representan secuencias de disparos que devuelven la red a un marcado anterior, es decir, los ciclos completos de procesamiento.

- IT1: T0 T1 T2 T3 T9 — ciclo completo de un pago con tarjeta de crédito/débito. La transacción es admitida, ruteada al flujo de tarjeta, autorizada por el issuer, capturada y liquidada.
- IT2: T0 T4 T5 T9 — ciclo completo de un pago de alto riesgo. La transacción es admitida, ruteada al flujo de alto riesgo donde el motor antifraude consulta al gateway en tiempo real, y es liquidada.
- IT3: T0 T6 T7 T8 T9 — ciclo completo de una transferencia bancaria. La transacción es admitida, ruteada al flujo de transferencia, validada la cuenta destino, ejecutada en la red bancaria y liquidada.

Clases propuestas para la implementación:

- Main: Instancia Logger, PetriNet, la política y el Monitor. Crea los hilos según SEGMENTS_SETUP, los inicia y espera a que T9 se dispare MAX_INVARIANTS veces. Al finalizar imprime los contadores de T1, T4 y T6.
- PetriNet: Contiene el marcado, la matriz de incidencia y el vector de alphas (tiempos mínimos). Implementa la lógica de sensibilización, verificación temporal y disparo de transiciones, además de validar los invariantes de plaza tras cada disparo.
- Monitor: Usa un semáforo mutex justo (FIFO) y un semáforo privado por transición. Aplica el patrón signal-and-exit, tras un disparo exitoso, consulta la política para despertar al próximo hilo sin liberar el mutex.
- MonitorInterface: Interfaz con un único método booleano, fireTransition(int transition).
- Segment: Cada instancia tiene asignada una secuencia de transiciones que dispara en loop llamando al monitor, hasta que éste retorna false (condición de parada).
- PolicyInterface: Interfaz con un único método entero, selectTransition(boolean[] enabledTransitions).
- PrioritizedPolicy: Siempre elige T4 si está habilitada; si no, delega en RandomPolicy.
- RandomPolicy: Elige aleatoriamente entre las transiciones habilitadas.
- Logger: Registra cada disparo en transitions_log.txt. Inicialmente se configuró un modo debug, el cual también escribe marcado, contadores, nombre del hilo y tiempo transcurrido en debug_transitions_log.txt, para verificar el correcto funcionamiento.

Además, se implementó un programa en Python, llamado [RegExTest.py](#), el cual lee el archivo transitions_log.txt y aplica expresiones regulares para verificar que las secuencias de disparos respeten los invariantes de transición de la red, validando el correcto funcionamiento.

Diagrama de clases:

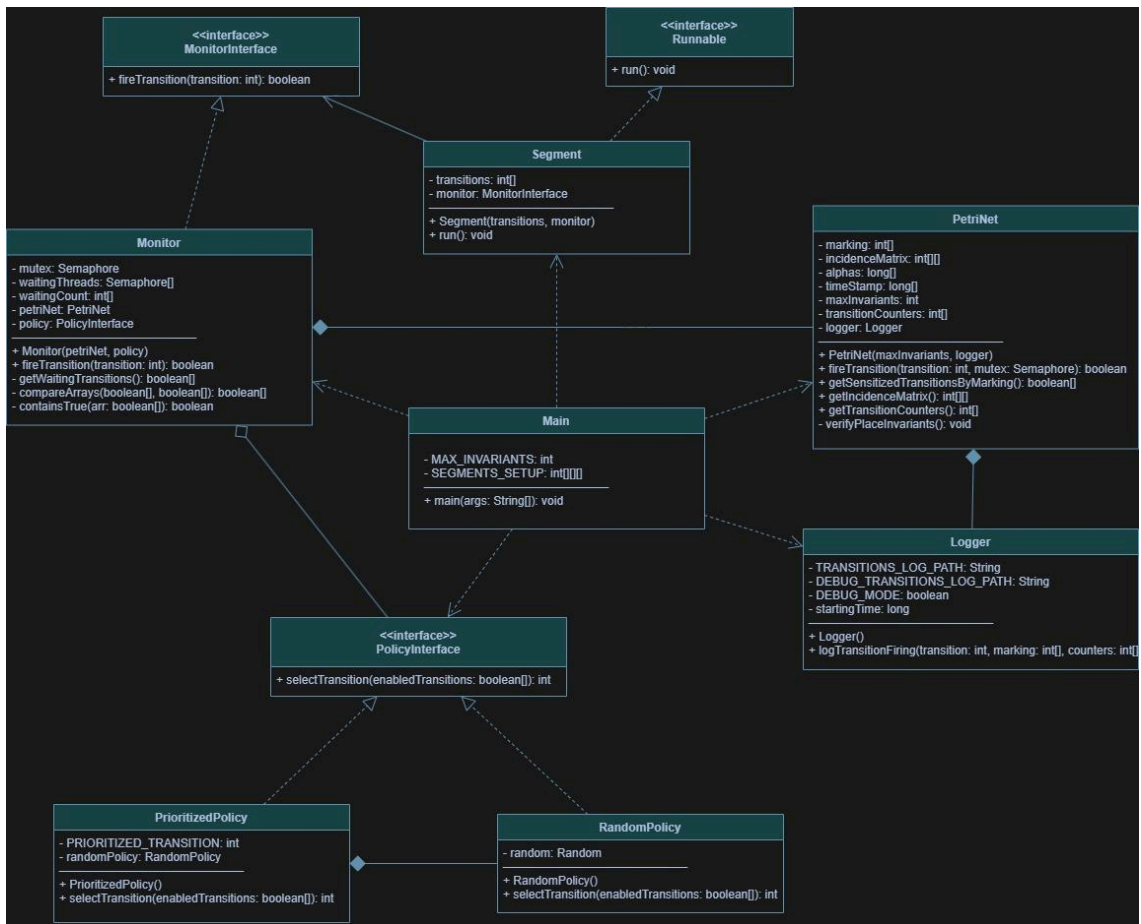
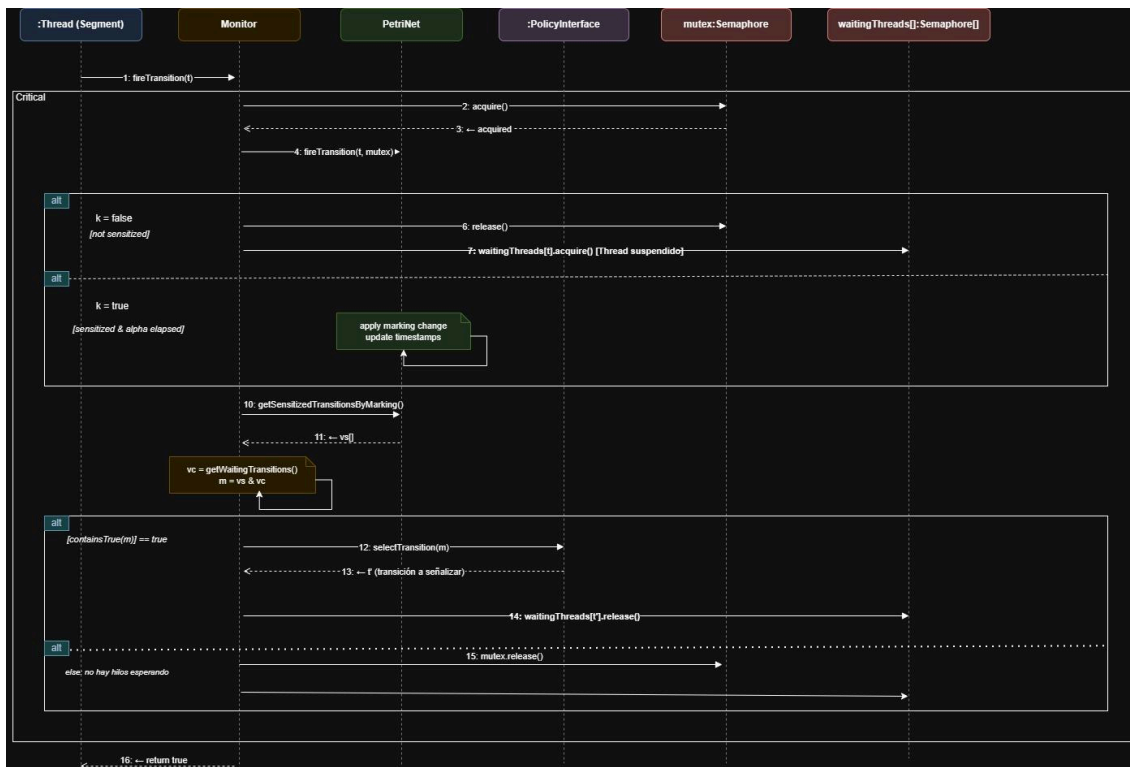


Diagrama de secuencia:



Cantidad de hilos:

Para determinar la cantidad máxima de hilos necesarios se aplicó el algoritmo de determinación de hilos por segmento. Primero se identificaron los segmentos de ejecución de la red a partir de sus invariantes de transición y de las estructuras de fork y join. En esta red se obtuvieron cinco segmentos principales: el segmento previo al fork, las tres ramas alternativas de ejecución y el segmento posterior al join.

Luego se analizó el marcado máximo posible de cada segmento, considerando las plazas de acción y descartando las plazas IDLE y de recursos compartidos. Los segmentos asociados a P1 y P9 pueden acumular hasta tres marcas, mientras que las ramas superior, central e inferior quedan limitadas por los recursos compartidos P7 y P8, por lo que requieren como máximo un hilo cada una.

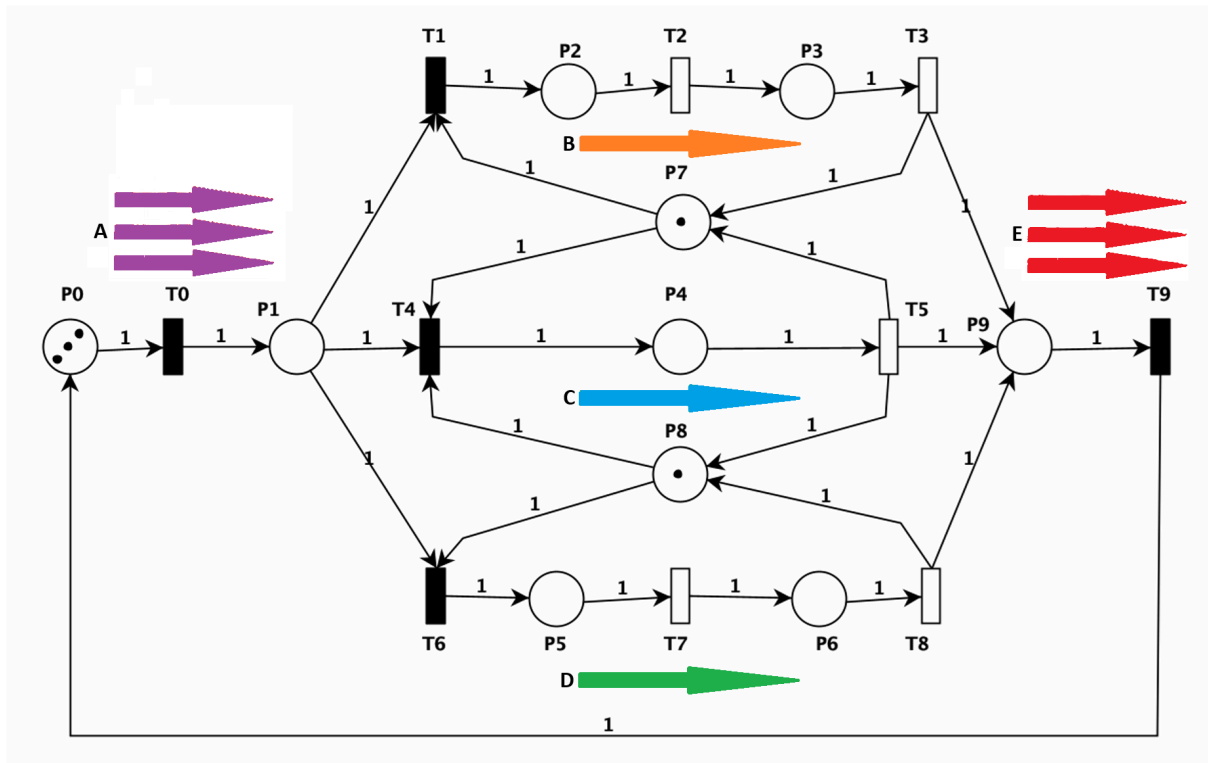
Por lo tanto, la cantidad máxima de hilos necesarios resulta:

$$[3 + 1 + 1 + 1 + 3 = 9]$$

Por lo tanto, podemos concluir que para esta Red de Petri se requieren como máximo 9 hilos de ejecución.

Sin embargo, en la práctica se verificó experimentalmente que utilizar 1 o 2 hilos produce tiempos de ejecución equivalentes. Esto se debe a la estructura de la red: los caminos alternativos del fork raramente convergen de forma simultánea en el join, por lo que las plazas P1 y P9 difícilmente acumulan más de un token al mismo tiempo durante la ejecución real.

Gráfico de responsabilidades de cada hilo:



Resultados obtenidos — Distribución por política:

Política priorizada:

Credit/Debit: 130
High-Risk: 3
Bank Transfer: 67
Total: 200 / 200
Tiempo total de ejecución: 26,11 s

Credit/Debit: 130
High-Risk: 2
Bank Transfer: 68
Total: 200 / 200
Tiempo total de ejecución: 24,06 s

Política aleatoria:

Credit/Debit: 132
High-Risk: 0
Bank Transfer: 68
Total: 200 / 200
Tiempo total de ejecución: 25,27 s

Credit/Debit: 132
High-Risk: 1
Bank Transfer: 67
Total: 200 / 200
Tiempo total de ejecución: 24,87 s

Los resultados muestran que el flujo de alto riesgo se completa muy pocas veces independientemente de la política, aunque la diferencia entre ambas es significativa.

Para que una transacción de alto riesgo se procese, T5 necesita que P7 y P8 estén disponibles de forma simultánea. Sin embargo, en condiciones normales de ejecución, P7 está casi siempre ocupado por el flujo de tarjeta y P8 por el de transferencia. Para que ambos queden libres al mismo tiempo, una transacción de tarjeta y una de transferencia deben finalizar en forma prácticamente simultánea, liberando sus recursos en la misma ventana temporal. Este evento es infrecuente y no predecible, ya que los dos flujos operan de forma completamente independiente.

- Con la política priorizada, cuando ocurre esta coincidencia y ambos recursos están disponibles al mismo tiempo, el hilo del flujo de alto riesgo tiene prioridad para adquirir el monitor. Esto le permite capturar esas ventanas de disponibilidad simultánea, resultando en 0 a 4 transacciones de alto riesgo por ejecución.
- Con la política aleatoria, esa misma ventana de disponibilidad existe, pero el hilo de alto riesgo compite en igualdad de condiciones con los otros flujos para adquirir el mutex. La probabilidad de que gane la competencia es menor, lo que resulta en 0 o 1 transacciones de alto riesgo por ejecución.

Análisis temporal:

1. Justificación de tiempos asignados:

Los tiempos fueron elegidos para reflejar la complejidad relativa de cada flujo según la semántica del modelo.

- Flujo de tarjeta de crédito/débito ($T2 + T3 = 180\text{ms}$): el flujo consta de dos etapas sobre el gateway (P7). Primero se solicita la autorización al emisor y se espera su respuesta ($T2 = 100\text{ms}$); luego se realiza la captura de fondos ($T3 = 80\text{ms}$), que al ejecutarse sobre la misma sesión ya establecida no requiere una nueva consulta.
- Flujo de transferencia bancaria ($T7 + T8 = 340\text{ms}$): el flujo opera de forma exclusiva sobre el motor antifraude (P8). Al ser operaciones irreversibles, requieren un control de riesgo más estricto que las de tarjeta, lo que se refleja en un tiempo total mayor. La validación de la cuenta destino ($T7 = 200\text{ms}$) es más lenta que la autorización de tarjeta porque la irreversibilidad exige un análisis más riguroso. La ejecución en la red bancaria ($T8 = 140\text{ms}$) también requiere más tiempo porque la confirmación es definitiva.
- Flujo de alto riesgo ($T5 = 400\text{ms}$): es la transición más costosa del sistema, y la única que requiere ambos recursos de forma simultánea (P7 y P8). El motor antifraude consulta señales en tiempo real al gateway mientras realiza el scoring. Al combinar las dos operaciones más pesadas del sistema sobre recursos críticos distintos, su tiempo supera los otros dos flujos.

2. Análisis cuantitativo:

- Tiempo teórico secuencial:

En un sistema de un solo hilo, cada transacción se procesa una a la vez, sin acciones concurrentes.

$$T_{\text{seq}} = N_c \times 180\text{ms} + N_h \times 400\text{ms} + N_t \times 340\text{ms}$$

Con los valores típicos obtenidos ($N_c = 130$, $N_h = 3$, $N_t = 67$)

$$\begin{aligned} T_{\text{seq}} &= 130 \times 180 + 3 \times 400 + 67 \times 340 \\ &= 23400 + 1200 + 22780 \\ &= 47400\text{ms} \end{aligned}$$

- Tiempo teórico concurrente:

El sistema tiene dos recursos independientes: P7 (gateway de tarjetas) y P8 (motor antifraude). Como operan de forma autónoma, pueden atender transacciones en paralelo, P7 procesa tarjetas mientras P8 procesa transferencias al mismo tiempo.

Para calcular el tiempo total, se analiza la carga acumulada sobre cada recurso compartido. Al operar bajo exclusión mutua, cada recurso atiende sus transacciones de una a la vez, por lo que su tiempo total es la suma de todos los tiempos de las transacciones que lo requieren.

P7 es utilizado por el flujo de tarjeta y por el flujo de alto riesgo. P8 es utilizado por el flujo de transferencia y también por el flujo de alto riesgo. Las transacciones de alto riesgo aparecen en la carga de ambos recursos porque T5 los requiere de forma simultánea: mientras T5 se ejecuta, ninguno de los dos puede ser utilizado por otro flujo.

La carga total de cada línea es:

$$\begin{aligned} \text{TP7} &= N_c \times 180\text{ms} + N_h \times 400\text{ms} \\ &= 130 \times 180 + 3 \times 400 \\ &= 24600\text{ms} \end{aligned}$$

$$\begin{aligned} \text{TP8} &= N_t \times 340\text{ms} + N_h \times 400\text{ms} \\ &= 67 \times 340 + 3 \times 400 \\ &= 23980\text{ms} \end{aligned}$$

Como ambas líneas trabajan en paralelo, el tiempo total del sistema lo determina la última en finalizar: 24,6s

La diferencia entre ambas cargas es de apenas 620ms, lo que indica que el sistema está prácticamente balanceado: ningún recurso permanece esperando mientras el otro trabaja. Ambos recursos operan a plena capacidad de forma simultánea, y el tiempo de ejecución total está determinado por esa ejecución paralela en sí misma — no por la saturación de un único componente. Agregar más hilos no reduciría el tiempo: el límite está en los recursos, no en la concurrencia.

Podemos concluir diciendo que la concurrencia reduce el tiempo de procesamiento en aproximadamente un 50% respecto al modelo secuencial equivalente.

Verificación de invariantes de plaza:

Luego de cada disparo de la red, se invoca el método `verifyPlaceInvariants`, el cual calcula la suma de tokens en cada conjunto de plazas que conforman un invariante y la compara con el valor esperado. Si la suma no coincide, se lanza una excepción que detiene la ejecución. De esta forma, se garantiza que los invariantes de plaza se mantienen en todo momento durante la simulación: el sistema nunca permite que un disparo deje la red en un estado inconsistente sin detectarlo de inmediato.

Verificación de invariantes de transición:

Para verificar el cumplimiento de los invariantes de transición, se aplican expresiones regulares sobre el log: cada regex representa una secuencia válida de disparos que corresponde a un invariante de transición. Si el patrón se encuentra la cantidad correcta de veces en el log, el invariante se considera verificado.

[illegible]

Conclusión:

El trabajo permitió modelar e implementar un sistema de procesamiento de pagos mediante Redes de Petri y un monitor de concurrencia en Java. Las propiedades verificadas con PIPE garantizan que el sistema opera de forma correcta y predecible. Los invariantes de plaza y transición, verificados en tiempo de ejecución y mediante expresiones regulares respectivamente, confirmaron la consistencia del modelo en toda ejecución.

El análisis temporal mostró que la concurrencia reduce el tiempo de procesamiento en un 50% respecto al modelo secuencial, con tiempos medidos de aproximadamente 25 segundos, consistentes con el valor teórico. El cuello de botella no es la cantidad de hilos sino la exclusión mutua sobre los recursos compartidos P7 y P8.

Respecto a las políticas, la distribución entre tarjeta y transferencia bancaria es estable en ambos casos. El flujo de alto riesgo es más frecuente con política priorizada, pero en ambos casos es infrecuente dado que requiere que P7 y P8 queden libres de forma simultánea, lo cual rara vez ocurre.